Redes convolucionales e interpretabilidad

Big Data y Machine Learning para Economía Aplicada

Ignacio Sarmiento-Barbieri

Universidad de los Andes

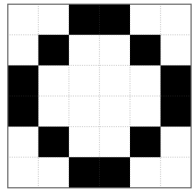
Agenda

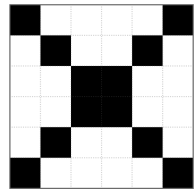
1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

Recap

- Segun el teorema de aproximacion univeral: Una red con una capa suficientemente ancha aproxima cualquier función continua sobre un compacto;
  - Deep networks puede lograrlo con muchas menos unidades.
- La clave esta enla arquitectura, que pasa si usamos una arquitectura que explote las caracteristicas delos datos

Motivación





Source: Starmer (2024), *The StatQuest Illustrated Guide to Neural Networks and AI*, cap. 6, https://github.com/StatQuest/signa

- Queremos un modelo que, dada una imagen de 6×6 píxeles, diga si es una **O** o una **X**.
- Que pasa si usamos una red densa?

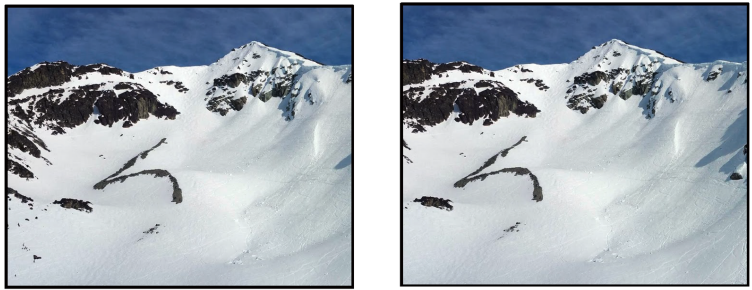
Problema 1: muchos parámetros

- Una imagen de 6×6 tiene 36 píxeles: cada neurona de la primera capa oculta necesita 36 pesos.
- Una imagen de 100×100 necesita 10.000 pesos por neurona.
- Una imagen RGB de 224×224 tiene $224 \times 224 \times 3 = 150.528$ inputs. Una sola capa oculta del mismo tamaño: $150.528^2 \approx 22.700$ millones de pesos.
- Cuanto más grande la imagen y más neuronas tengamos, más parámetros hay que aprender, y más datos hacen falta para aprenderlos.

Problema 2: sensibilidad a pequeños desplazamientos

- Entrenamos la red para reconocer una X centrada en la imagen.
- Corremos la X un píxel a la derecha. Para la red densa, cada píxel es un input distinto con su propio peso: **nada garantiza** que siga reconociéndola.
- Lo que queremos es que la respuesta no cambie cuando la imagen se desplaza (invarianza) y que el mapa de "dónde está" se desplace con ella (equivarianza).

Invarianza



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

- Si desplazamos la imagen, la respuesta a "¿es una montaña?" no debería cambiar: $f(Tx) = f(x)$.

Equivarianza



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

- Si desplazamos la imagen, el mapa de "dónde está el sofá" debería desplazarse igual: $f(Tx) = Tf(x)$.

Problema 3: no aprovecha las relaciones espaciales

- Los píxeles cercanos suelen estar relacionados: forman una misma región de color, o un borde.
- Al aplanar la imagen, la red densa no incorpora esa estructura: el píxel 7 y el píxel 8 son dos inputs sin más relación entre sí que el 7 y el 700.
- ¿Cómo lo sabemos? Con un experimento.

Problema 3: no aprovecha las relaciones espaciales

Alteremos el orden



photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

Solucion?

- Una arquitectura que
  1. reduzca la cantidad de parámetros,
  2. aproveche que los píxeles vecinos están relacionados,
  3. sea robusta ante pequeños desplazamientos.

Convolutional Neural Networks

- Las redes neuronales volvieron a la primera línea alrededor de 2010 gracias a sus éxitos en clasificación de imágenes.
- La familia responsable: las **redes convolucionales** (CNN). LeCun et al. (1998) las usaron para leer dígitos; Krizhevsky, Sutskever & Hinton (2012) ganaron ImageNet con AlexNet.
- Resuelven los tres problemas con dos restricciones:
  - **Conectividad local**: cada unidad mira solo una vecindad pequeña del input.
  - **Pesos compartidos**: los mismos pesos se usan en todas las posiciones.
- Las capas convolucionales son equivariantes a traslaciones (con stride 1 y salvo en los bordes); el *pooling* agrega invarianza parcial.

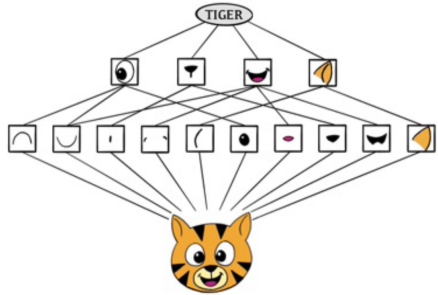
Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

Filtros que recorren la imagen

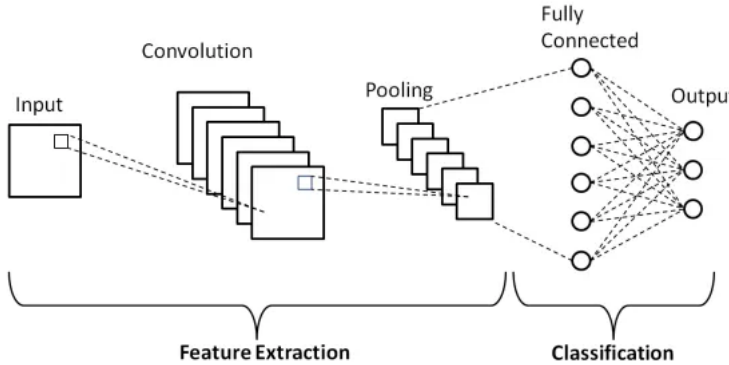
- Las **Convolutional Neural Networks** (CNN) resuelven los tres problemas con **filtros** que recorren la imagen y extraen características locales.
- En lugar de aprender una conexión independiente para cada píxel y cada neurona, usan filtros pequeños (por ejemplo, de 3×3) cuyos pesos **se comparten** a lo largo de toda la imagen.
- El mismo filtro busca el mismo rasgo en todas las posiciones: pocos parámetros, vecindad, y el rasgo se detecta esté donde esté.

Filtros que recorren la imagen

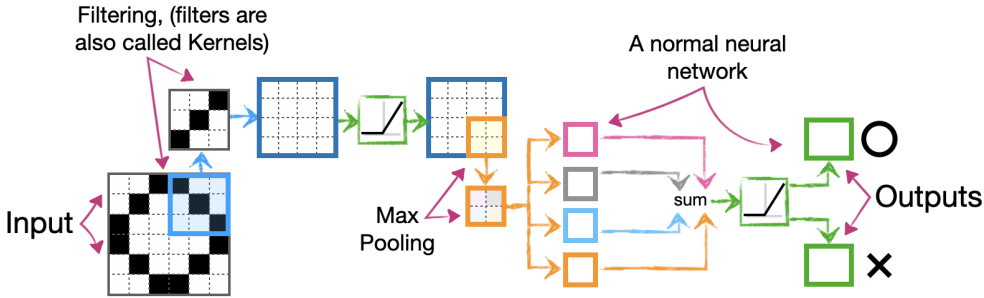


Source: James, Witten, Hastie & Tibshirani (2021), *An Introduction to Statistical Learning*, 2.ª ed., cap. 10

Architecture of a Convolutional Neural Network



- Bloques de convolución + pooling extraen rasgos; al final se “aplana” y unas capas densas clasifican.
- A medida que avanzamos, la imagen se achica y el número de canales crece.



Source: Starmer (2024), *The StatQuest Illustrated Guide to Neural Networks and AI*, cap. 6, https://github.com/StatQuest/signa

Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

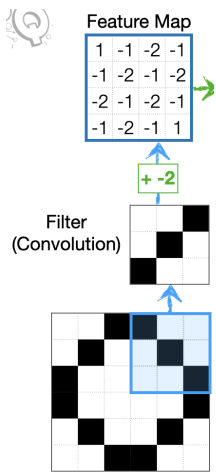
Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

Convolution: extracción de características

- El primer paso es aplicar un **filtro** (o *kernel*) sobre la imagen original.
- El filtro, por ejemplo de 3×3, recorre la imagen y hace una operación sobre cada región de píxeles. Cada posición produce un número que se guarda en un **feature map**.
- El procedimiento:
  1. Colocar el filtro sobre una región de la imagen.
  2. Multiplicar cada valor del filtro por el píxel correspondiente.
  3. Sumar todos los productos (producto punto).
  4. Agregar un término de sesgo (*bias*).
  5. Guardar el resultado en el feature map y desplazar el filtro a la siguiente región.

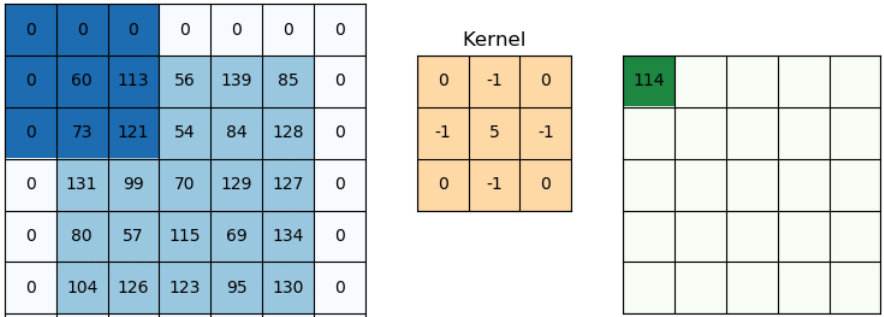
Convolution (O o X)



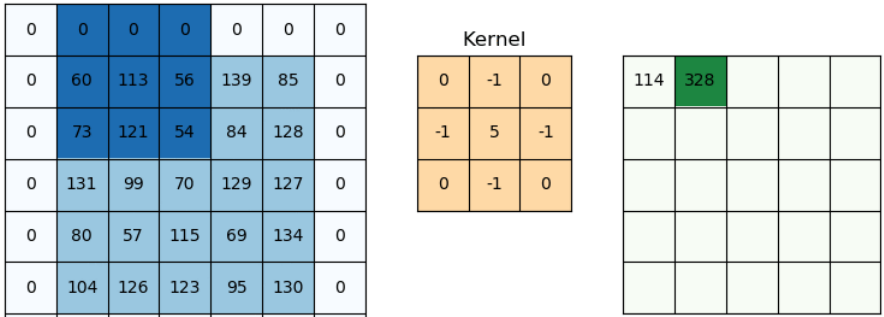
Source: Starmer (2024), *The StatQuest Illustrated Guide to Neural Networks and AI*, cap. 6, https://github.com/StatQuest/signa

- Filtro de 3×3 con unos en la antidiagonal y ceros en el resto; sesgo $b = -2$.
- En la esquina superior izquierda, el filtro cubre tres píxeles encendidos: $1 + 1 + 1 - 2 = 1$.
- En la esquina superior derecha, resaltada en la figura, cubre uno: $1 - 2 = -1$.
- Con una imagen de 6×6 y un filtro de 3×3 (*stride* 1, sin *padding*) hay 4×4 posiciones: el feature map es de 4×4.

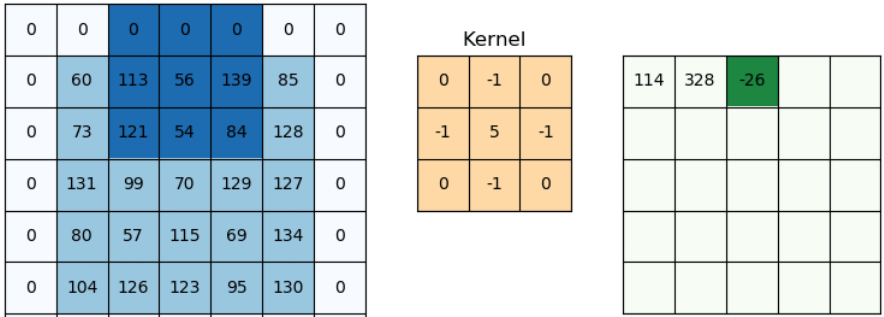
2D Convolution



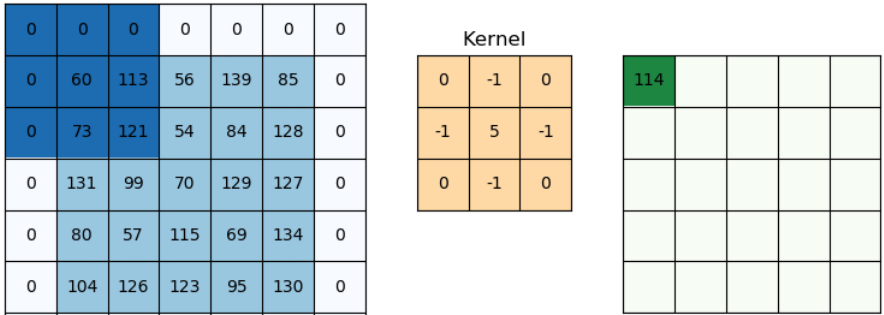
2D Convolution



2D Convolution



2D Convolution



Filtros conocidos y filtros aprendidos

- En procesamiento de imágenes los filtros se eligen a mano:

Suavizar (*blur*)

$$\frac{1}{9}\begin{pmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{pmatrix}$$

Bordes verticales

$$\begin{pmatrix}-1 & 0 & 1\\ -1 & 0 & 1\\ -1 & 0 & 1\end{pmatrix}$$

- El *blur* es un promedio simple de los vecinos.
- El de bordes es una diferencia: vale cero en zonas uniformes y es grande donde la intensidad cambia de izquierda a derecha.
- Una CNN **aprende** los filtros para la tarea: sus valores son parámetros, estimados por backpropagation como cualquier otro peso.

Filtros conocidos: Instagram y Lenna



photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

La matemática

- Cada entrada del feature map es

$$z_{i,j} = \sum_r \sum_c W_{r,c} X_{i+r,j+c} + b$$

donde W contiene los pesos del filtro, X es la imagen y b el sesgo.

- Es un producto punto entre el filtro y la región de la imagen que tiene debajo, repetido en cada posición.

- Tamaño del output con imagen de $n \times n$, filtro de $k \times k$, *stride* s y sin *padding*:

$$\left\lfloor \frac{n-k}{s} \right\rfloor + 1 \qquad \text{(en el ejemplo: } \tfrac{6-3}{1} + 1 = 4\text{)}$$

2D Convolution

- En una imagen, el kernel es una matriz pequeña, típicamente 3×3:

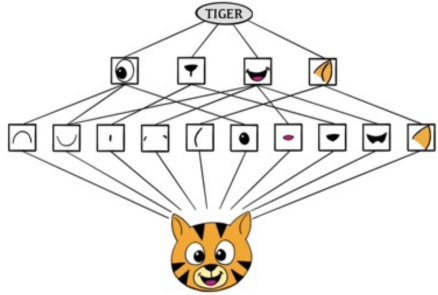
$$h_{ij} = g\left(\beta + \sum_{r=1}^{3}\sum_{c=1}^{3} \omega_{rc}\, x_{i+r-2,\, j+c-2}\right)$$

- donde ω son los pesos W, β el sesgo b y g la activación.
- El kernel recorre la imagen y mira una vecindad de 3×3 por vez: analiza **rasgos locales**.
- Con una imagen a color, el input tiene 3 canales (RGB) y el kernel es de $3 \times 3 \times 3$.

¿Qué hacen los filtros?

- El filtro detecta **patrones locales**: bordes, líneas, formas.
- En procesamiento de imágenes los pesos W se eligen a mano. En una CNN son **parámetros**: se aprenden durante el entrenamiento por backpropagation, como cualquier otro peso.
- Una capa tiene muchos filtros, y las capas se apilan: los filtros de la segunda capa combinan los rasgos que encontró la primera.

Una jerarquía de rasgos



Source: James, Witten, Hastie & Tibshirani (2021), *An Introduction to Statistical Learning*, 2.ª ed., cap. 10

- Las primeras capas detectan bordes y manchas; las siguientes combinan esos rasgos en partes (ojos, orejas); las últimas, en objetos.

Detalles: kenel size, stride, padding, dilation

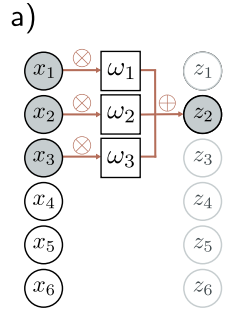
Convolución en 1D

- Empecemos con un input unidimensional $x = (x_1, \ldots, x_I)$: una serie, una fila de píxeles.
- Una convolución con un **kernel** de tamaño 3, $\omega = (\omega_1, \omega_2, \omega_3)$:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

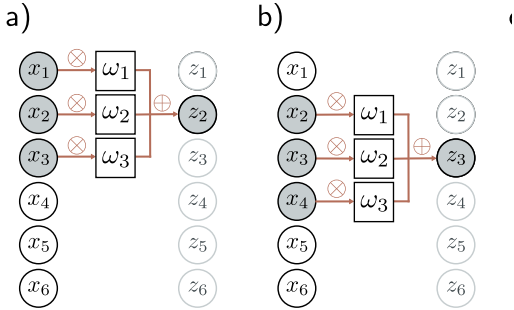
- Cada output es una **suma ponderada de los vecinos** de x_i.
- Los pesos ω son los mismos para todo i: el kernel "se desliza" sobre el input.

Convolution with kernel size 3



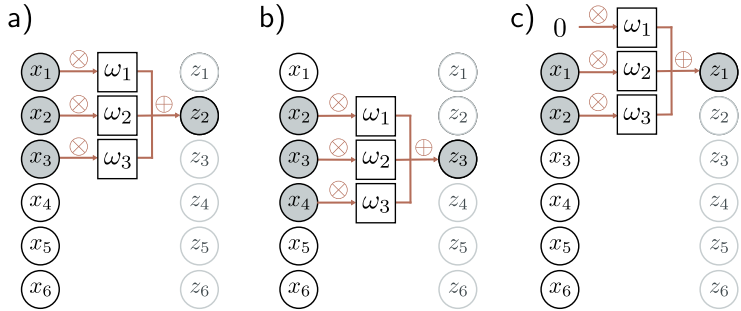
Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

Convolution with kernel size 3



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

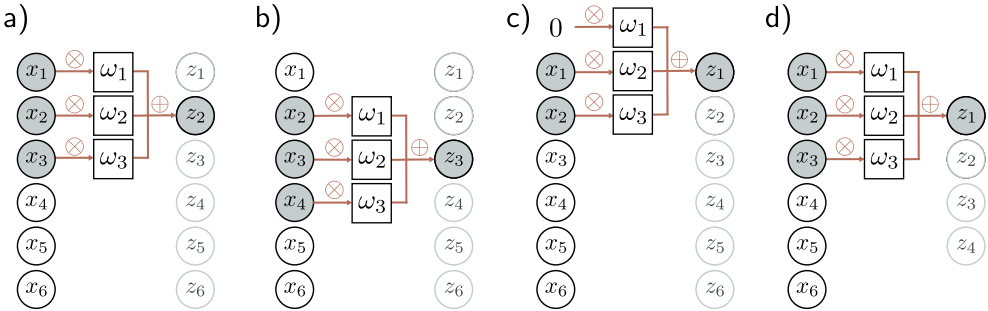
Zero padding



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

- En los bordes falta un vecino: se completa con ceros (*padding* "same"), y el output tiene el mismo tamaño que el input.

“Valid” convolutions



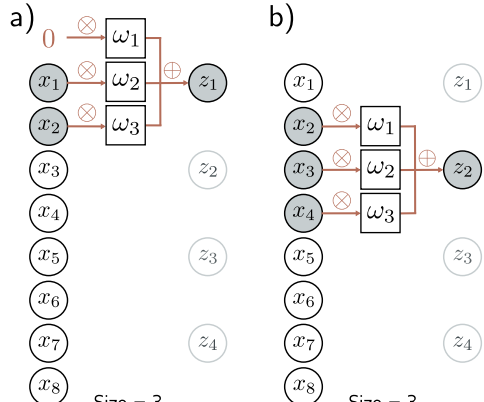
Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

- O se calculan solo las posiciones donde el kernel cabe entero (*valid*): el output pierde un elemento por lado.

Stride, kernel size, and dilation

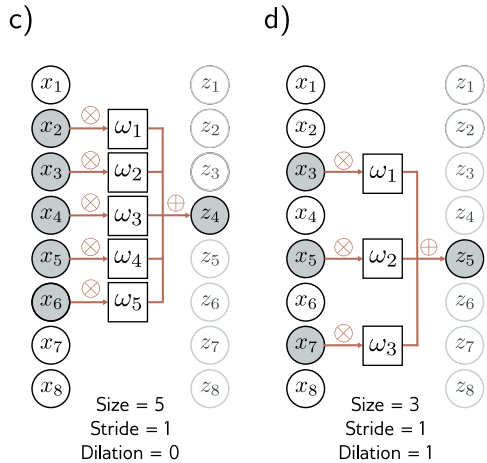
- **Stride**: el kernel avanza s posiciones entre un output y el siguiente.
  - El output es más chico que el input.
- **Kernel size**: cuántos inputs pondera cada output.
  - Combina información de un área más grande...
  - ...pero un kernel de tamaño 5 usa 5 parámetros.
- **Dilation** (*atrous*): intercalar ceros dentro del kernel.
  - Combina información de un área más grande con menos parámetros.

Stride, kernel size, and dilation



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

Stride, kernel size, and dilation



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

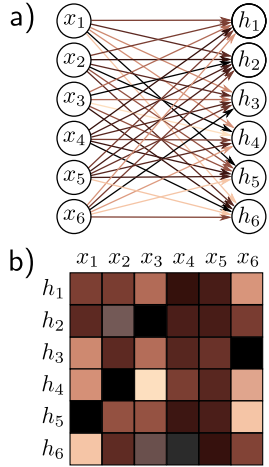
Convolutional layers

- Una **capa convolucional** convoluciona el input, suma un intercepto y pasa el resultado por una activación:

$$h_i = g\left(\beta + \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}\right)$$

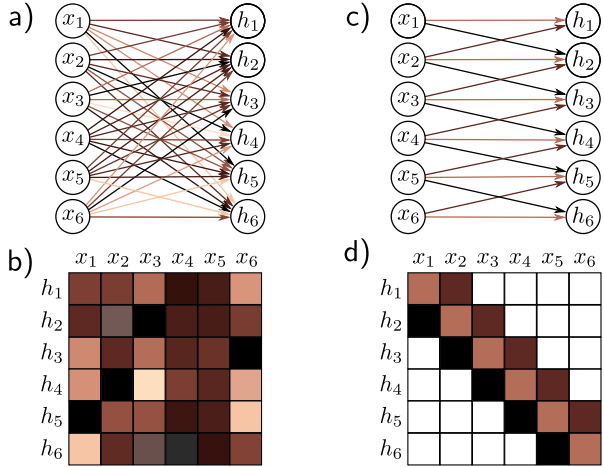
- Con un input de tamaño 6 y padding:
  - Capa densa: $6 \times 6 + 6 = 42$ parámetros.
  - Capa convolucional: $3 + 1 = 4$ parámetros, **sin importar el tamaño del input**.

Fully Connected vs. convolutional layers



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

Fully Connected vs. convolutional layers



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

Una capa convolucional es una capa densa restringida

- Escribamos la convolución como $z = \Omega x$. Con kernel de tamaño 3 y padding:

$$\Omega = \begin{pmatrix} \omega_2 & \omega_3 & 0 & 0 & 0 & 0 \\ \omega_1 & \omega_2 & \omega_3 & 0 & 0 & 0 \\ 0 & \omega_1 & \omega_2 & \omega_3 & 0 & 0 \\ 0 & 0 & \omega_1 & \omega_2 & \omega_3 & 0 \\ 0 & 0 & 0 & \omega_1 & \omega_2 & \omega_3 \\ 0 & 0 & 0 & 0 & \omega_1 & \omega_2 \end{pmatrix}$$

- Dos restricciones sobre una matriz densa: **ceros** fuera de la banda (localidad) y **el mismo valor** en cada diagonal (pesos compartidos).
- Restringir un modelo implica menos parámetros, menos varianza, mas sesgo. Si la restricción es correcta para el problema, el sesgo que agrega es poco.

Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

ReLU: la función de activación

- Después de la convolución, aplicamos una activación a cada valor del feature map:

$$\text{ReLU}(z) = \max(0, z)$$

- Los valores negativos se vuelven cero; los positivos se mantienen.

Antes de ReLU

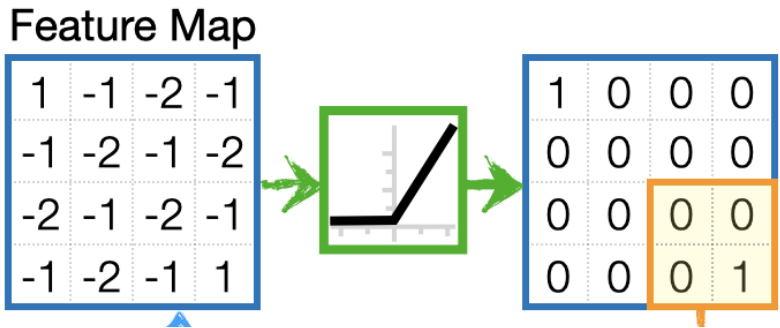
$$\begin{pmatrix} 1 & -1 & -2 & -1 \\ -1 & -2 & -1 & -2 \\ -2 & -1 & -2 & -1 \\ -1 & -2 & -1 & 1 \end{pmatrix} \longrightarrow$$

Después de ReLU

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- ¿Para qué? Introduce **no linealidad**: sin ella, apilar convoluciones sería apilar operaciones lineales, y el resultado seguiría siendo lineal.

ReLU en el ejemplo



Source: Starmer (2024), *The StatQuest Illustrated Guide to Neural Networks and AI*, cap. 6, https://github.com/StatQuest/signa

- El feature map de 4×4 pasa elemento por elemento por la ReLU. Sobreviven solo las dos posiciones donde el filtro encontró su patrón completo.

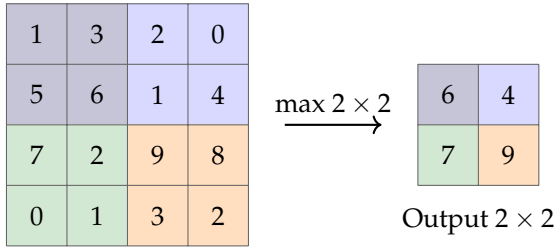
Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

Pooling Layers

- Las capas de *pooling* reducen la dimensión de la imagen resumiendo regiones.
- **Max pooling**: toma bloques de 2×2 que no se superponen y se queda con el máximo de cada uno.
- Reduce la imagen a la mitad en cada dirección y agrega algo de **invarianza**: si el rasgo se mueve un píxel dentro del bloque, el máximo no cambia.
- No tiene parámetros.

Pooling Layers



$$y_{ij} = \max_{(m,n) \in \text{bloque}(i,j)} x_{mn}$$

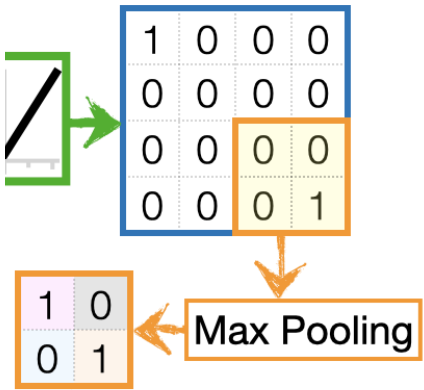
Pooling: dos objetivos

- Por ejemplo, con una región de 2×2:

$$\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix} \quad \longrightarrow \quad 6$$

- Con **max pooling** conservamos la activación más fuerte de cada región.
- Esto cumple dos objetivos:
  - **Reducir la dimensionalidad**: menos valores para representar las características extraídas.
  - **Tolerar pequeños desplazamientos**: si la característica se mueve un poco dentro de la región, el máximo no cambia.
- También existe **average pooling**: en lugar del máximo, el promedio.

Pooling en el ejemplo



Source: Starmer (2024), *The StatQuest Illustrated Guide to Neural Networks and AI*, cap. 6, https://github.com/StatQuest/signa

- Bloques de 2×2 que **no se solapan**: el bloque avanza de a 2 (*stride* = tamaño del bloque).
- $4 \times 4 \to 2 \times 2$: de 16 valores quedan 4.
- El 1 de la esquina superior izquierda y el 1 de la inferior derecha sobreviven. Si cualquiera de los dos se hubiera corrido un píxel dentro de su bloque, el output sería el mismo.

Pooling: al cuaderno



photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

Flatten y clasificación con una red densa

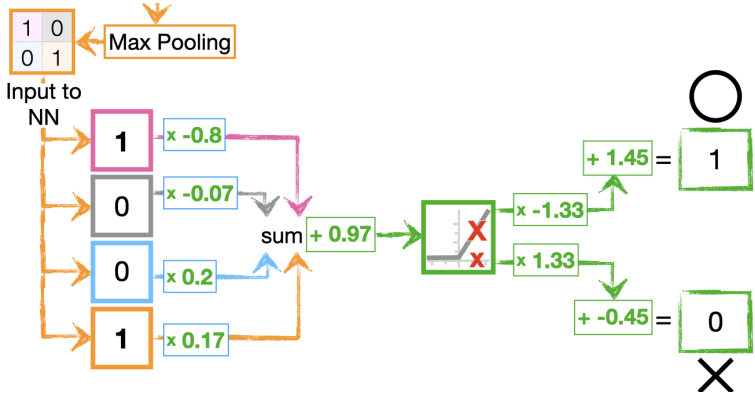
- Después de convolution, ReLU y pooling tenemos una representación compacta de la imagen. Falta convertirla en una predicción.

- **Flatten**: el feature map reducido se transforma en un vector.

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

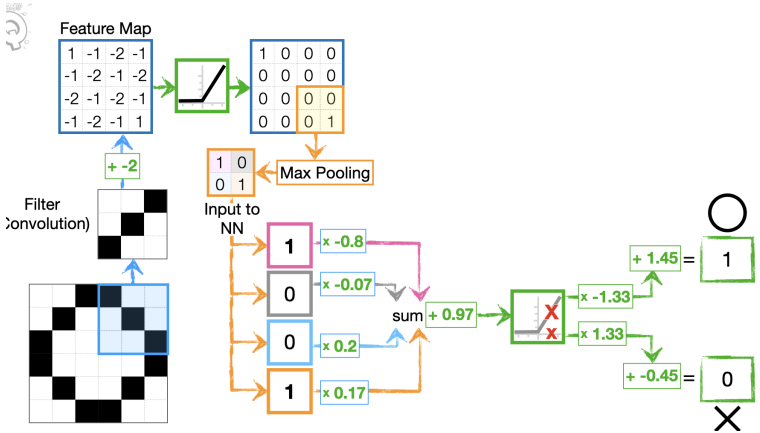
- Ese vector es el input de una red densa (*fully connected*), que combina las características con pesos, sesgos y activaciones y produce dos salidas: una para O y otra para X.

Flatten y clasificación en el ejemplo



Source: Starmer (2024), *The StatQuest Illustrated Guide to Neural Networks and AI*, cap. 6, https://github.com/StatQuest/signa

La red completa



Source: Starmer (2024), *The StatQuest Illustrated Guide to Neural Networks and AI*, cap. 6, https://github.com/StatQuest/signa

- De 36 píxeles a 4 valores antes de la parte densa: primero extraer rasgos y reducir, después clasificar.

Ejemplo: MNIST

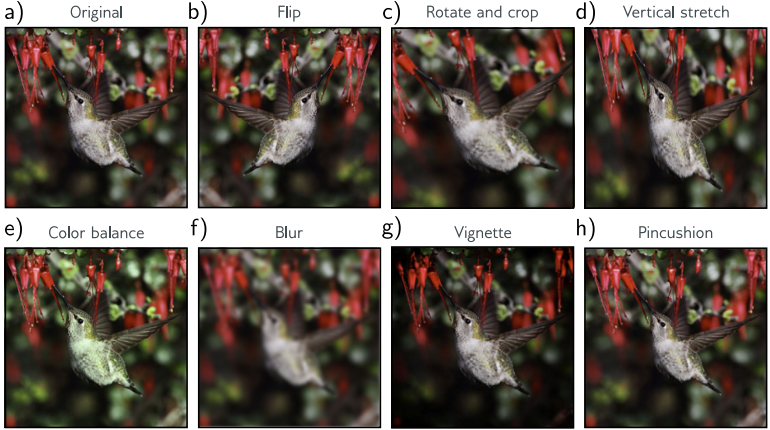


photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

Data augmentation



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

Data augmentation

- Generamos nuevas observaciones de entrenamiento transformando las que tenemos: reflejos, rotaciones, recortes, cambios de color, **desplazamientos**.
- La etiqueta no cambia: un colibrí reflejado sigue siendo un colibrí; una O corrida un píxel sigue siendo una O.
- Es otra forma de **regularizar**: le decimos a la red qué transformaciones no deben cambiar la predicción.
- Ojo con las transformaciones que sí cambian la etiqueta: un 6 rotado 180° es un 9.

Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

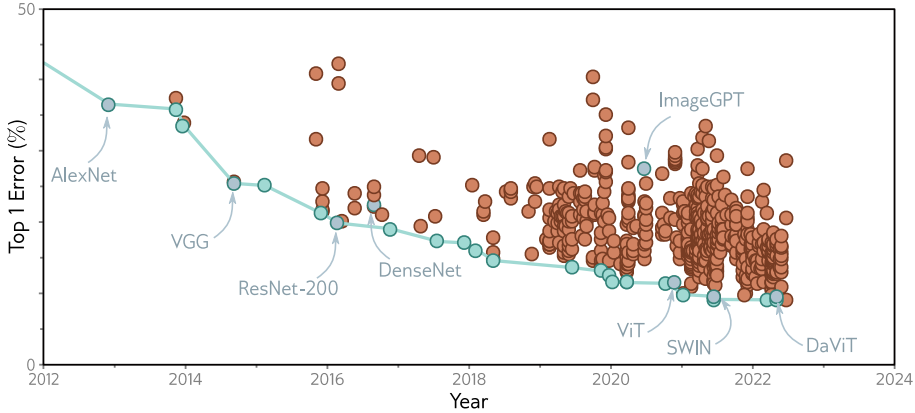
ImageNet

- **ImageNet** es una enorme base de datos de imágenes clasificadas en categorías: perros, gatos, carros, aves, edificios, ...
- Más de 14 millones de imágenes etiquetadas. La versión usada en competencias, **ImageNet-1K**, tiene aproximadamente 1,2 millones de imágenes de entrenamiento en 1.000 clases.

ImageNet

- **ImageNet** es una enorme base de datos de imágenes clasificadas en categorías: perros, gatos, carros, aves, edificios, ...
- Más de 14 millones de imágenes etiquetadas. La versión usada en competencias, **ImageNet-1K**, tiene aproximadamente 1,2 millones de imágenes de entrenamiento en 1.000 clases.
- ¿Por qué fue importante? Porque permitió comparar arquitecturas de visión artificial **bajo las mismas condiciones**.
- La competencia: *ImageNet Large Scale Visual Recognition Challenge* (ILSVRC), 2010–2017. Cada año, distintos modelos intentaban clasificar correctamente las mismas imágenes.

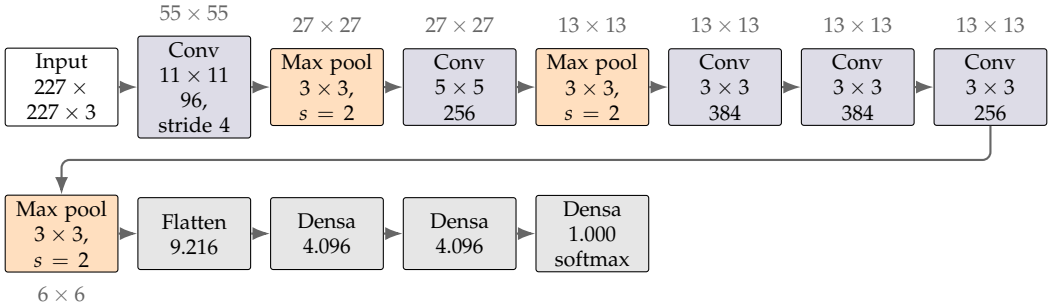
ImageNet: el problema que cambió todo



Source: Prince (2023), *Understanding Deep Learning*, https://udlbook.github.io/udlbook/

- 1,2 millones de imágenes, 1.000 clases. AlexNet (2012) bajó el error de manera drástica; después vinieron VGG, ResNet, ...

AlexNet (Krizhevsky, Sutskever & Hinton, 2012)



AlexNet: por qué funcionó

- Alrededor de 60 millones de parámetros, casi todos en las capas densas. Entrenada en 1,2 millones de imágenes durante una semana en dos GPUs.
- Bajó el error top-5 de ImageNet de 26% a 15%. El segundo puesto de 2012 no usaba redes.
- La arquitectura (cuántos filtros, de qué tamaño, dónde poner el pooling) **surgió de la experimentación**. Los años siguientes (VGG, ResNet) son más capas y mejores trucos para entrenarlas, sobre el mismo esqueleto.

Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

¿Qué miró la red?

- Ya sabemos cómo construir modelos capaces de clasificar imágenes.
- Pero ¿cómo podemos explicar qué características fueron importantes para **una predicción específica**?

¿Qué miró la red?

- Ya sabemos cómo construir modelos capaces de clasificar imágenes.
- Pero ¿cómo podemos explicar qué características fueron importantes para **una predicción específica**?
- En una regresión lineal la respuesta está en los coeficientes: β_j dice cuánto aporta cada unidad de x_j, para cualquier observación.
- En una CNN con 225.034 parámetros, o en un bosque con 500 árboles, no hay un β_j que mirar.
- El modelo predice bien y no sabemos bien por qué.

Dos ejes para clasificar explicaciones

- **Intrínseca vs. post-hoc**
  - Intrínseca: el modelo es interpretable por construcción (lineal, un árbol chico).
  - Post-hoc: el modelo ya está entrenado y es una caja negra; la explicación se calcula después.
- **Local vs. global**
  - Local: ¿por qué el modelo predijo *esto* para *esta* observación?
  - Global: ¿qué variables usa el modelo, en promedio, sobre todos los datos?
- Shapley values son una explicación **post-hoc y local**; agregándolas obtenemos una global.

Un ejemplo con tres variables

- Olvidemos las redes por un momento. Supongamos que **conocemos** el modelo:

$$f(x) = 10 + 2x_1 + 3x_2 - x_3 + 4\,x_1x_2$$

  con x_1, x_2, x_3 independientes y de media cero.

- Observación a explicar: $x = (1, 1, 2)$.

$$f(x) = 10 + 2 + 3 - 2 + 4 = 17, \qquad E[f(X)] = 10$$

- La predicción está 7 por encima del promedio. ¿Cuánto de esos 7 le corresponde a cada variable?

Un ejemplo con tres variables

- Olvidemos las redes por un momento. Supongamos que **conocemos** el modelo:

$$f(x) = 10 + 2x_1 + 3x_2 - x_3 + 4\,x_1x_2$$

  con x_1, x_2, x_3 independientes y de media cero.

- Observación a explicar: $x = (1, 1, 2)$.

$$f(x) = 10 + 2 + 3 - 2 + 4 = 17, \qquad E[f(X)] = 10$$

- La predicción está 7 por encima del promedio. ¿Cuánto de esos 7 le corresponde a cada variable?

- Los términos lineales son fáciles: 2, 3 y -2. Suman 3.

- El problema es la interacción $4x_1x_2 = 4$: no pertenece a x_1 ni a x_2 sola.

La función de valor $v(S)$

- Pensemos las variables como **jugadores** y la predicción como un **pago** a repartir.
- Para una coalición S de variables, $v(S)$ es lo que la predicción se aleja del promedio si solo fijamos las variables en S:

$$v(S) = E_{X_{-S}}[f(x_S, X_{-S})] - E[f(X)]$$

- Las variables fuera de S se integran con su distribución **marginal** (si son independientes, es lo mismo que condicionar). Como tienen media cero, desaparecen:

S	$\emptyset$	$\{1\}$	$\{2\}$	$\{3\}$	$\{1,2\}$	$\{1,3\}$	$\{2,3\}$	$\{1,2,3\}$
$v(S)$	0	2	3	-2	9	0	1	7

- Por ejemplo, $v(\{1,2\}) = 2 \cdot 1 + 3 \cdot 1 + 4 \cdot 1 \cdot 1 = 9$: la interacción aparece solo cuando x_1 y x_2 están juntas.

El reparto depende del orden

- Una idea natural: las variables entran de a una y cada una se lleva lo que **agrega** al entrar (su contribución marginal).
- Orden $x_1 \to x_2 \to x_3$:

$$x_1:\ v(\{1\}) - v(\emptyset) = 2, \qquad x_2:\ v(\{1,2\}) - v(\{1\}) = 9 - 2 = 7,$$
$$x_3:\ v(\{1,2,3\}) - v(\{1,2\}) = 7 - 9 = -2$$

El reparto depende del orden

- Una idea natural: las variables entran de a una y cada una se lleva lo que **agrega** al entrar (su contribución marginal).
- Orden $x_1 \to x_2 \to x_3$:

$$x_1:\ v(\{1\}) - v(\emptyset) = 2, \qquad x_2:\ v(\{1,2\}) - v(\{1\}) = 9 - 2 = 7,$$
$$x_3:\ v(\{1,2,3\}) - v(\{1,2\}) = 7 - 9 = -2$$

- Orden $x_2 \to x_1 \to x_3$:

$$x_2:\ v(\{2\}) - v(\emptyset) = 3, \qquad x_1:\ v(\{1,2\}) - v(\{2\}) = 9 - 3 = 6, \qquad x_3:\ -2$$

- Los dos repartos suman 7, pero son distintos: **la interacción se la lleva la última de x_1, x_2 en entrar**.
- Ningún orden es el correcto. Shapley (1953): **promediar sobre todos los órdenes**.

Promediar sobre todos los órdenes

Orden de entrada	Contribución de x_1	de x_2	de x_3
$1 \to 2 \to 3$	2	7	-2
$1 \to 3 \to 2$	2	7	-2
$2 \to 1 \to 3$	6	3	-2
$2 \to 3 \to 1$	6	3	-2
$3 \to 1 \to 2$	2	7	-2
$3 \to 2 \to 1$	6	3	-2
Promedio $= \phi_j$	**4**	**5**	$\mathbf{-2}$

- $\phi_1 + \phi_2 + \phi_3 = 7 = f(x) - E[f(X)]$: el reparto es **exacto**.
- x_3 no interactúa con nadie y recibe exactamente su término lineal, -2.
- La interacción de 4 se partió en mitades: x_1 recibe $2 + 2$, x_2 recibe $3 + 2$.

El valor de Shapley

- Para n variables y la variable i:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \underbrace{\frac{|S|!\,(n - |S| - 1)!}{n!}}_{\text{peso de la coalición } S} \underbrace{\left[v(S \cup \{i\}) - v(S)\right]}_{\text{contribución marginal de } i \text{ a } S}$$

- Es un **promedio ponderado de las contribuciones marginales** de i a través de todas las coaliciones que se pueden formar sin ella.

El valor de Shapley

- Para n variables y la variable i:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \underbrace{\frac{|S|!\,(n-|S|-1)!}{n!}}_{\text{peso de la coalición } S} \underbrace{\left[v(S \cup \{i\}) - v(S)\right]}_{\text{contribución marginal de } i \text{ a } S}$$

- Es un **promedio ponderado de las contribuciones marginales** de i a través de todas las coaliciones que se pueden formar sin ella.
- ¿De dónde sale el peso? De contar órdenes. Hay $n!$ órdenes de entrada. Para que al entrar i encuentre **exactamente** a los miembros de S:
  - los $|S|$ miembros de S deben entrar antes: $|S|!$ formas,
  - los $n - |S| - 1$ restantes deben entrar después: $(n - |S| - 1)!$ formas.
- El peso es la **fracción de los órdenes** en los que i se encuentra con S al entrar.

Por qué esta forma de repartir y no otra

- Shapley (1953) probó que es el **único** reparto que cumple cuatro propiedades:

1. **Eficiencia**: $\sum_i \phi_i = v(N) = f(x) - E[f(X)]$. Se reparte todo, ni más ni menos.
2. **Simetría**: si dos variables aportan lo mismo a toda coalición, reciben lo mismo.
3. **Dummy**: una variable que no cambia la predicción en ninguna coalición recibe 0.
4. **Aditividad**: el Shapley de $v + w$ es el Shapley de v más el de w. Para un bosque, se puede calcular árbol por árbol y promediar.

SHAP: SHapley Additive exPlanations

- Lundberg & Lee (2017) llevan la idea a cualquier modelo de ML. La explicación de una predicción es **aditiva**:

$$f(x) = \phi_0 + \sum_{j=1}^{p} \phi_j, \qquad \phi_0 = E[f(X)]$$

- ϕ_0 es el **valor de referencia**: lo que el modelo predice sin saber nada de x; se estima con el promedio de las predicciones sobre datos de fondo.
- Para un modelo lineal sin interacciones, $\phi_j = \beta_j (x_j - E[X_j])$: el coeficiente, medido desde la referencia.

SHAP: SHapley Additive exPlanations

- El problema es calcularlo: 2^p coaliciones y una esperanza por cada una. Tres atajos:
  - **Monte Carlo** por órdenes aleatorios (Štrumbelj & Kononenko, 2014), usados en las redes.
  - **KernelSHAP**: regresión lineal ponderada sobre coaliciones muestreadas, con pesos derivados de los axiomas.
  - **TreeSHAP**: exacto y rápido para árboles, bosques y boosting.

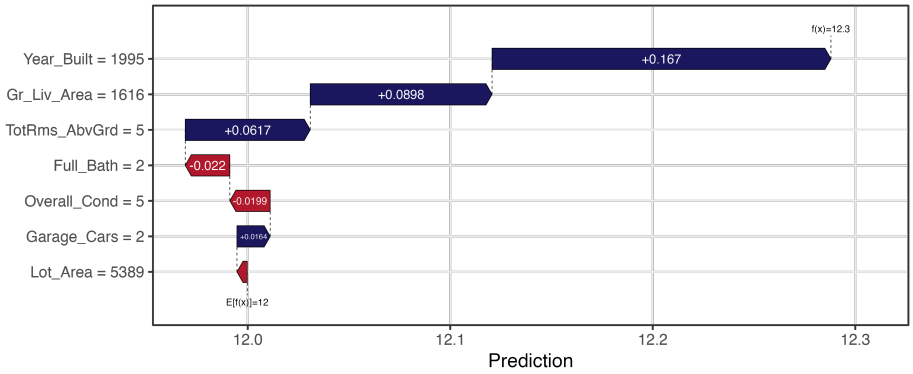
Cómo se leen las contribuciones

- $\phi_j > 0$: el valor x_j de **esta** observación empuja la predicción **por encima** de la referencia ϕ_0.
- $\phi_j < 0$: la empuja por debajo.
- Las unidades son las de f: pesos, log-precio, probabilidad.

Cómo se leen las contribuciones

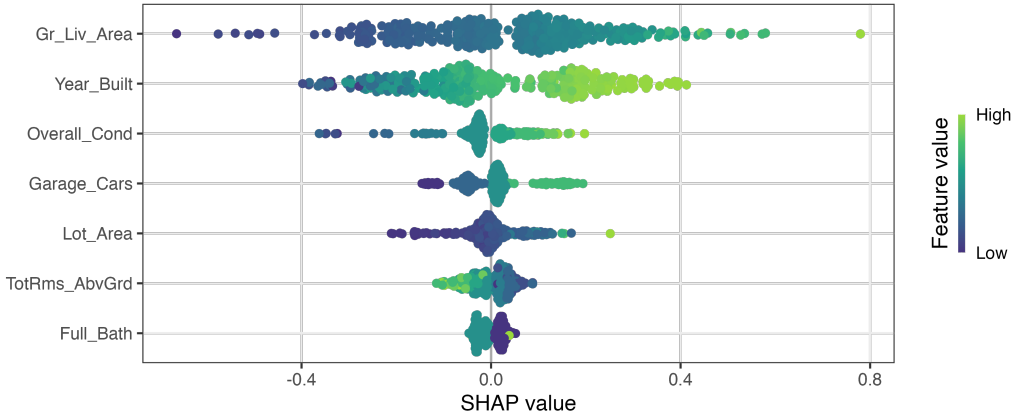
- $\phi_j > 0$: el valor x_j de **esta** observación empuja la predicción **por encima** de la referencia ϕ_0.
- $\phi_j < 0$: la empuja por debajo.
- Las unidades son las de f: pesos, log-precio, probabilidad.
- Tres cosas que ϕ_j **no** es:
  - No es el cambio en la predicción si sacamos x_j del modelo y reentrenamos.
  - No es una derivada ni un contrafactual: $\phi_j > 0$ no dice que subir x_j suba f.
  - No es absoluto: cambia si cambia la referencia. Es la respuesta a "¿por qué $f(x)$ y no ϕ_0?"

Waterfall: una casa de Ames



- XGBoost para $\log(\text{precio})$ con 7 variables; KernelSHAP con 200 casas de comparacion (con $p = 7$ enumera las 128 coaliciones; TreeSHAP sería la vía rápida).
- Partimos de ϕ_0 y cada barra suma o resta hasta llegar a $f(x)$. $+0,167$ en log-precio son $\approx +18\%$.

Beeswarm: todas las casas



Limitaciones

- **Depende del valor de referencia.** La explicación responde "¿por qué $f(x)$ y no ϕ_0?"
  - Cambiar la referencia cambia la respuesta: todas las casas vs solo las de la zona;
  - No hay una referencia correcta: toca elegirla

Limitaciones

- **Depende del valor de referencia.** La explicación responde "¿por qué $f(x)$ y no ϕ_0?"
  - Cambiar la referencia cambia la respuesta: todas las casas vs solo las de la zona;
  - No hay una referencia correcta: toca elegirla

- **Variables correlacionadas.** Al integrar las variables fuera de S con su distribución marginal creamos observaciones que no existen: una casa de 50 m^2 con 6 baños. El modelo se evalúa donde nunca vio datos.

Limitaciones

- **Costo computacional.** 2^p coaliciones. Con 10 variables son 1.024; con 30, mil millones; con 784 píxeles, inviable.
  - Shortcuts, Monte Carlo, KernelSHAP o TreeSHAP, y por eso, salvo con pocas variables o con TreeSHAP, una explicación SHAP es una **estimación** con error de muestreo.
- **No es un modelo de la realidad.** Explica el modelo, no el fenómeno: si la CNN mira el fondo de la foto, SHAP dirá que el fondo importa, y tendrá razón sobre la red.
- **Predicción vs. inferencia (again)** ϕ_j no es un efecto causal de x_j sobre y. Es un reparto contable de la predicción de un modelo entrenado para predecir.

Agenda

1. Motivación
2. ¿Qué hacen las CNN?
3. Cómo lo hacen?
   - Convolution
   - ReLU
   - Pooling
   - Flatten y clasificación
4. Entrenar una CNN
5. ImageNet
6. Interpretabilidad: Shapley values y SHAP
7. Next

Recap

1. La red densa no usa el orden de los píxeles, tiene demasiados parámetros y no es robusta a desplazamientos.
2. Una convolución es una suma ponderada de los vecinos con pesos que se comparten en todas las posiciones: una capa densa con restricciones.
3. Convolution $\rightarrow$ ReLU $\rightarrow$ pooling extraen rasgos y reducen la dimensión; flatten y una red densa clasifican.
4. Las restricciones bajan drásticamente los parámetros de cada capa convolucional y dan equivarianza a traslaciones; el pooling agrega invarianza parcial, y data augmentation ayuda a complementar la invarianza restante.
5. ImageNet y AlexNet: la arquitectura surge de experimentar.
6. Shapley reparte $f(x) - \phi_0$ entre las variables promediando sobre todos los órdenes; SHAP lo lleva a cualquier modelo. Depende de la referencia y no es un efecto causal.

Next

- Próxima clase:
  - Datos espaciales y dependencia espacial.
  - Por qué la validación cruzada aleatoria falla con datos espaciales.
  - Redes para datos geográficos: CNN sobre imágenes satelitales.